MY SERVER SETUP
PROBLEMS & SOLUTIONS

A Personal Troubleshooting Journal — Real Problems I Hit, How I Fixed Them, What I Learned

Topics: SSH · systemd · Network Interfaces · DHCP · USB Tethering · Cloudflare Tunnel

April 2026 | Ubuntu Server | Personal Lab Notes

Overview

This document is a personal journal of every real problem I ran into while setting up remote access to my Ubuntu server from my laptop. Each entry records exactly what went wrong, the root cause, how I fixed it, and the underlying concept.

Goal: Access my Ubuntu server terminal from my laptop over SSH, then extend to work from anywhere.

# Problem	Concept Learned
1. SSH not installed on server	SSH client vs server (sshd), TCP port 22
2. SSH service not running after install	systemd — start vs enable vs status
3. Don't know the server's IP address	Network interfaces & ip a command
4. Server has NO network (interfaces DOWN)	Interface UP/DOWN states, USB tethering
5. SSH connection refused / timeout	TCP ports, firewall (ufw), debugging layers
6. No IP assigned after USB tether connected	DHCP — dhclient, IP lease negotiation
7. SSH works on LAN but not from outside	NAT, private IPs, Cloudflare Tunnel

PROBLEM #1 SSH Not Installed on Server

✗ SYMPTOM — WHAT HAPPENED

Ran ssh ultron@<ip> from laptop. Got 'Connection refused'. Assumed SSH was ready to go — the server had never had it installed.

⚠ ROOT CAUSE

Ubuntu Server minimal installs do NOT ship with OpenSSH server. The 'ssh' command on the laptop is the SSH CLIENT. The server needs the SSH SERVER daemon (sshd) to accept connections. Without sshd, port 22 has nothing listening on it, so the connection is refused at the TCP level immediately.

✓ HOW IT WAS FIXED

Installed the openssh-server package on the Ubuntu server using apt. This placed the sshd binary and its config (/etc/ssh/sshd_config) onto disk. Then started and enabled the service.

◆ CONCEPT LEARNED

SSH is a CLIENT-SERVER protocol. Your laptop runs ssh (the client). Your server must run sshd (the daemon). They are two different programs from the same package. The client initiates the connection to TCP port 22. The server listens on port 22. Without sshd, there is nothing listening — the OS rejects the TCP connection instantly with RST, which appears as 'Connection refused'.

$ Commands Used

```
# Run ON THE SERVER (not the laptop)
sudo apt update
sudo apt install openssh-server -y

# Confirm sshd binary is present
which sshd
sshd --version

# Check what is listening on port 22 after install
ss -tlnp | grep 22
```

PROBLEM #2 SSH Service Not Running After Install

✗ SYMPTOM — WHAT HAPPENED

Installed openssh-server but SSH still refused connections. The package was on disk but the service was not started.

⚠ ROOT CAUSE

Installing a package with apt only copies files to disk. It does NOT automatically start the service. On Ubuntu, sshd needs to be explicitly started with systemctl. Also, 'start' only runs it NOW — without 'enable', the service will not start after a reboot, so the same problem will return next boot.

✓ HOW IT WAS FIXED

Used 'sudo systemctl enable --now ssh' which both enables (boot persistence) and starts (immediate) the service in one command. Verified it was running with 'systemctl status ssh' — looked for 'active (running)'.

◆ CONCEPT LEARNED

systemd is the init system managing all background services on modern Linux. 'systemctl start' = run the service right now. 'systemctl enable' = make it auto-start at every boot. 'systemctl status' = check if it is alive. These are independent actions. You almost always need BOTH start AND enable for a server service.

$ Commands Used

```
# Start SSH now AND enable at boot (do both together)
sudo systemctl enable --now ssh

# Verify — look for: 'active (running)'
sudo systemctl status ssh

# See exactly what port sshd is listening on
ss -tlnp | grep sshd

# Useful systemctl commands for reference
sudo systemctl start ssh      # start now
sudo systemctl stop ssh       # stop now
sudo systemctl restart ssh    # restart
sudo systemctl enable ssh     # auto-start at boot
sudo systemctl disable ssh    # remove from boot
```

PROBLEM #3 Did Not Know the Server's IP Address

✗ SYMPTOM — WHAT HAPPENED

SSH requires typing ssh user@IP_ADDRESS. Did not know what IP the server had been assigned. Could not connect without knowing this.

⚠ ROOT CAUSE

Every device on a network has an IP address attached to a network interface. The IP is assigned either manually (static) or automatically by DHCP (the router gives it). On a headless server with no monitor-based GUI, you must use a terminal command to find it.

✓ HOW IT WAS FIXED

Ran 'ip a' on the server. This lists every network interface and shows the 'inet' line with the IPv4 address. Found the active interface (eno1 or usb0) and read the inet address (e.g. 192.168.42.167).

◆ CONCEPT LEARNED

Network interfaces are named connection points: eno1 is a wired Ethernet port, wlo1 is Wi-Fi, usb0 is a USB tethered phone. Each can have an IP. 'ip a' (ip address show) dumps them all. The inet field shows IPv4 in CIDR notation: 192.168.42.167/24 — the /24 means subnet mask 255.255.255.0. The IP before the slash is what you type in SSH.

$ Commands Used

```
# Show ALL interfaces and IPs
ip a
# OR shorthand
ip addr

# Show only IPv4, cleaner output
ip -4 addr show

# Quick one-liner — just the IPs
hostname -I

# What to look for in ip a output:
# 2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP>
#     inet 192.168.1.5/24 brd 192.168.1.255
#          ^^^^^^^^^^^^^
#          YOUR IP — use this in SSH

# Then from LAPTOP:
ssh ultron@192.168.1.5
```

PROBLEM #4 Server Had NO Active Network — Both Interfaces Were DOWN

✗ SYMPTOM — WHAT HAPPENED

Ran 'ip a' on the server and saw: eno2 state DOWN and wlo1 state DOWN. No inet line visible anywhere. SSH was impossible — nothing to connect to.

⚠ ROOT CAUSE

DOWN means no physical link. For eno2 (Ethernet) it means no cable is plugged in. For wlo1 (Wi-Fi) it means not associated with any access point. Without a link-layer connection, DHCP cannot run, no IP is assigned, and no TCP connection can be made. The server was completely isolated from any network.

✓ HOW IT WAS FIXED

Two options. Option A: plug an Ethernet cable from the router to the server eno2 port — simple, reliable. Option B: enable USB tethering on a smartphone, connect it via USB cable to the server. The phone presents itself as a USB network adapter (usb0 interface) and runs its own DHCP server. Server gets internet via the phone's mobile data. Used USB tethering since no Ethernet cable was available.

◆ CONCEPT LEARNED

Interface states: DOWN = no physical signal, no data can flow. UP = physical link active. LOWER_UP = link layer connected (cable has signal). For USB tethering specifically: the phone creates a virtual network adapter over USB using the RNDIS protocol. Linux sees it as usb0. The phone's hotspot runs a DHCP server in the 192.168.42.x range. The server gets an IP from the phone. The phone does NAT and shares its mobile data.

$ Commands Used

```
# Check all interface states
ip link show
# 'state DOWN' = no connection, 'state UP' = connected

# ── USB Tethering Setup ──
# 1. Connect phone USB cable to server
# 2. On phone: Settings → Hotspot & Tethering → USB Tethering ON
# 3. Check dmesg on server to see new USB device
dmesg | tail -20
# Should show: rndis_host registered 'usb0'

# 4. Find the new interface
ip link show
# New interface usb0 should appear

# 5. Bring it up
sudo ip link set usb0 up

# 6. Request an IP from the phone via DHCP
sudo dhclient usb0

# 7. Verify IP was assigned
ip addr show usb0
# Should show: inet 192.168.42.xxx/24
```

PROBLEM #5 SSH 'Connection Refused' or 'Timed Out' Even With Network

✗ SYMPTOM — WHAT HAPPENED

After getting network via USB tethering, SSH from laptop still failed. 'Connection refused' on some attempts, 'timed out' on others. Two different error messages, two different causes.

⚠ ROOT CAUSE

'Connection refused' = TCP packet reached the server but nothing listening on port 22. Cause: sshd not running. 'Connection timed out' = TCP packet never arrived at the server. Cause: wrong IP

address entered, or firewall blocking port 22, or laptop and server on different subnets with no route between them.

✓ HOW IT WAS FIXED

Diagnosed layer by layer. First confirmed server IP with 'ip a'. Then tested basic reachability with 'ping'. Then checked if port 22 was open with 'nc -zv'. Then checked sshd status. Then checked ufw firewall. Found sshd had stopped — restarted it. Found ufw was active but had no rule for SSH — added 'sudo ufw allow ssh'.

◆ CONCEPT LEARNED

SSH over TCP needs every layer to work. (1) Physical: cable or Wi-Fi connected. (2) IP: both devices have IPs on reachable networks. (3) Routing: a path exists between them. (4) Firewall: port 22 allowed through ufw/iptables. (5) Application: sshd is running. 'Connection refused' = layers 1-4 OK, layer 5 failed. 'Timed out' = one of layers 1-4 failed. Always debug bottom-up.

$ Commands Used

```
# Step 1: Confirm server IP (on SERVER)
ip addr show

# Step 2: From LAPTOP — test basic reachability
ping -c 4 192.168.42.167
# Fails = routing/network problem
# Works = go to step 3

# Step 3: From LAPTOP — test port 22 specifically
nc -zv 192.168.42.167 22
# 'Connection refused' = sshd not running
# 'succeeded' = sshd is up, SSH will work

# Step 4: On SERVER — check sshd
sudo systemctl status ssh
sudo systemctl start ssh   # if not running

# Step 5: On SERVER — check firewall
sudo ufw status
sudo ufw allow ssh         # if active and blocking

# Step 6: On SERVER — confirm something listens on 22
ss -tlnp | grep 22
```

PROBLEM #6 USB Interface Appeared But Had No IP Address

✗ SYMPTOM — WHAT HAPPENED

After enabling USB tethering, usb0 appeared in 'ip link show' but 'ip addr show usb0' had no inet line. No IP = SSH still impossible.

⚠ ROOT CAUSE

An interface appearing does NOT mean it has an IP. They are two separate things. Having a link (physical/USB connection) only gives you layer 2 (MAC address level). An IP address is layer 3 and is obtained separately via DHCP. On a headless Ubuntu server with no NetworkManager or systemd-networkd configured for new interfaces, DHCP does not run automatically — you must trigger it manually.

✓ HOW IT WAS FIXED

Ran 'sudo dhclient usb0' to manually start the DHCP client on the usb0 interface. dhclient sent a DHCPDISCOVER broadcast. The phone's DHCP server responded with a DHCPOFFER of 192.168.42.167. Confirmed with 'ip addr show usb0' that inet now appeared.

◆ CONCEPT LEARNED

DHCP (Dynamic Host Configuration Protocol) is a 4-step process: DISCOVER (client broadcasts: 'anyone have an IP for me?') → OFFER (server offers: 'take 192.168.42.167') → REQUEST (client: 'yes I want that one') → ACKNOWLEDGE (server: 'it is yours for 1 hour'). This is called the DORA process. dhclient is the Linux DHCP client. The phone runs the DHCP server in hotspot mode. Without running dhclient, the server just sits at the link layer with no IP identity.

$ Commands Used

```
# Check if IP exists
ip addr show usb0
# If no 'inet' line → run dhclient

# Trigger DHCP manually
sudo dhclient usb0

# Watch the DORA handshake in verbose mode
sudo dhclient -v usb0
# You will see:
# DHCPDISCOVER on usb0
# DHCPOFFER from 192.168.42.129
# DHCPREQUEST for 192.168.42.167
# DHCPACK from 192.168.42.129

# Verify IP assigned
ip addr show usb0
# Should show: inet 192.168.42.167/24

# Test phone gateway
```

```
ping -c 3 192.168.42.129

# Test internet
ping -c 3 8.8.8.8
curl -I https://google.com
```

PROBLEM #7 SSH Works on LAN But Not from Outside the Network

✗ SYMPTOM — WHAT HAPPENED

SSH worked perfectly when laptop and server were on the same Wi-Fi. But from mobile data or a different network, connection always timed out. Could not reach the server from outside.

⚠ ROOT CAUSE

Local IPs (192.168.x.x) are private and not routable on the public internet. The home router does NAT: it maps all local devices to one public IP. Inbound connections from the internet cannot reach a private IP unless the router explicitly port-forwards them. Most home ISPs also block inbound connections by default. Even if port forwarding was set up, the public IP changes (dynamic IP).

✓ HOW IT WAS FIXED

Installed cloudflared on the server. The server runs 'cloudflared tunnel' which dials OUT to Cloudflare's edge over HTTPS (port 443, always allowed outbound). Cloudflare assigns a public URL. The laptop connects to that URL through Cloudflare, which routes the connection through the tunnel to the server. No port forwarding, no static IP, no router changes needed. Works from any network.

◆ CONCEPT LEARNED

NAT (Network Address Translation): the router rewrites outgoing packets to use its public IP. Return packets come back to the router which reverse-translates them to the right local device. This works for outbound. For inbound, the router does not know which local device a new connection is for — hence port forwarding is needed. Cloudflare Tunnel bypasses this entirely: the SERVER initiates an outbound persistent connection to Cloudflare. Since the server opened the connection, NAT tracks it and allows return traffic. The laptop's SSH goes to Cloudflare → rides the existing tunnel → exits at the server.

$ Commands Used

```
# ── Install cloudflared on the SERVER ──
curl -L https://pkg.cloudflare.com/cloudflare-main.gpg \
  | sudo tee /usr/share/keyrings/cloudflare-main.gpg > /dev/null

echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg]
  https://pkg.cloudflare.com/cloudflared focal main' \
```

```
  | sudo tee /etc/apt/sources.list.d/cloudflared.list

sudo apt update && sudo apt install cloudflared

# ── Quick tunnel — no account needed ──
cloudflared tunnel --url ssh://localhost:22
# Output: https://random-words.trycloudflare.com

# ── Connect from LAPTOP ──
ssh -o ProxyCommand='cloudflared access ssh --hostname random-
words.trycloudflare.com' ultron@localhost

# ── OR: make a persistent named tunnel (needs CF account) ──
cloudflared tunnel login
cloudflared tunnel create my-server
cloudflared tunnel route dns my-server ssh.yourdomain.com

# Run as a systemd service so it auto-starts
sudo cloudflared service install
sudo systemctl enable --now cloudflared
```

Key Concepts Summary

All the concepts I learned through these problems, in one place.

Concept	What It Means (Plain English)
SSH (sshd)	Remote terminal access protocol. Needs ssh CLIENT on laptop + sshd SERVER on the server. Uses TCP port 22. All traffic encrypted. Without sshd running on server, you cannot connect.
systemd / systemctl	Linux service manager. start = run now. enable = run at every boot. status = check health. Always use 'enable --now' for server daemons to do both at once.
Network Interface	Named connection point to a network (eno1 = ethernet, wlo1 = Wi-Fi, usb0 = USB tether). Must be in state UP AND have an IP address before SSH or any IP traffic works.
ip a / ip addr	Command to list all network interfaces and their IP addresses. The 'inet' line shows your IPv4. If no inet line exists, no IP is assigned yet.
Interface DOWN state	The physical link has no signal. No cable, no Wi-Fi, no USB. Data cannot flow. Must fix the physical connection before anything else works. This was my core blocker.

Concept	What It Means (Plain English)
DHCP + dhclient	Protocol for automatic IP assignment. Phone (in tether mode) is DHCP server. Run 'dhclient usb0' to ask for an IP. Without this step, interface has no IP even if cable is plugged in.
USB Tethering	Phone shares mobile data via USB. Linux sees phone as a USB network adapter (usb0). Phone does DHCP + NAT. Server gets internet without Wi-Fi or Ethernet router.
NAT & Private IPs	Router maps all local devices (192.168.x.x) to one public IP. Inbound connections from internet cannot reach private IPs without port forwarding. This is why SSH fails from outside LAN.
Cloudflare Tunnel	Server dials OUT to Cloudflare (bypasses NAT). Cloudflare gives a public URL. Laptop connects through Cloudflare to server. No port forwarding. No static IP. Works from anywhere.
Connection refused vs timeout	Refused = packet reached server but nothing listening on port 22 (sshd not running). Timeout = packet never arrived (wrong IP, firewall, or no network route between devices).

The SSH Debugging Checklist

When SSH fails, go through this checklist layer by layer, starting from the bottom. Each layer depends on the one below it.

```
$ SSH Debug Checklist — Run in Order

LAYER 1 — Physical Connection
  Is a cable plugged in? Is USB tethering ON? Is Wi-Fi connected?
  Check:  ip link show  →  look for 'state UP' or 'LOWER_UP'
  Fix:    plug cable / enable tethering on phone

LAYER 2 — IP Address
  Does the interface have an IP address?
  Check:  ip addr show  →  look for 'inet x.x.x.x'
  Fix:    sudo dhclient <interface>

LAYER 3 — Routing
  Is there a default route (path to the internet/gateway)?
  Check:  ip route show  →  look for 'default via x.x.x.x'
  Fix:    sudo ip route add default via <gateway> dev <interface>

LAYER 4 — Firewall
  Is port 22 allowed?
  Check:  sudo ufw status
  Fix:    sudo ufw allow ssh
```

```
LAYER 5 — sshd Running
  Is the SSH server daemon active on port 22?
  Check:  sudo systemctl status ssh
  Fix:    sudo systemctl enable --now ssh

LAYER 6 — Reachability
  Can the laptop actually reach the server? Same network?
  Check:  ping <SERVER_IP>  from laptop
  Fix:    Use Cloudflare Tunnel if on a different network

If all 6 layers pass → SSH WILL WORK.
```

— End of Journal —

Update this doc every time you hit a new problem.